

Servlet



¿Por qué apareció Servlet?

Antes de los servlets, las aplicaciones Web dinámicas utilizaban tecnologías como CGI (Common Gateway Interface).

Cada solicitud del usuario generaba un nuevo proceso en el servidor, lo que provocaba:

- ▶ mayor consumo de recursos
- ▶ bajo rendimiento
- ▶ poca escalabilidad

Los servlets surgieron para permitir la reutilización de componentes Java capaces de atender múltiples solicitudes.

¿Qué es un Servlet?

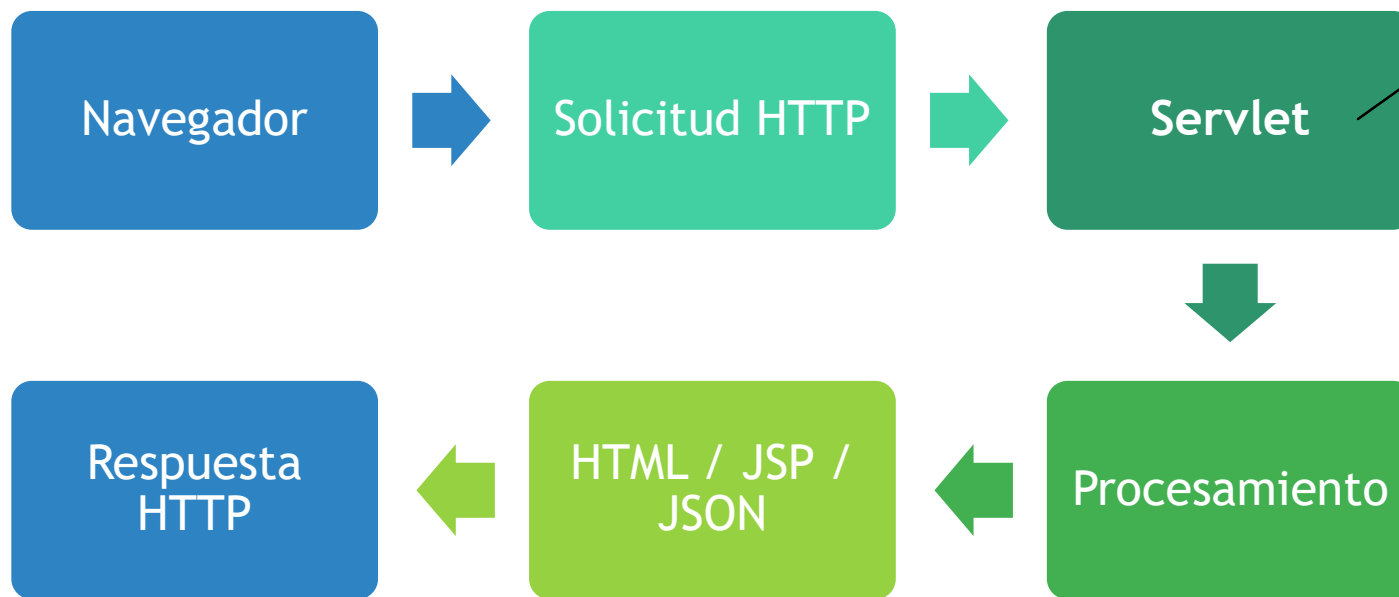
Un servlet es **una clase Java** que se ejecuta dentro de un servidor de aplicaciones y **permite procesar solicitudes HTTP** generadas por los clientes.

Su principal función es:

- ▶ recibir solicitudes
- ▶ procesar información
- ▶ generar respuestas dinámicas

Los servlets representan el componente encargado del procesamiento dentro de una aplicación Web Java.

Flujo de una aplicación Web



El servlet actúa como **intermediario** entre el navegador y la lógica de la aplicación.

¿Por qué no usar solo JSP?

Aunque JSP permite generar contenido dinámico, mezclar:

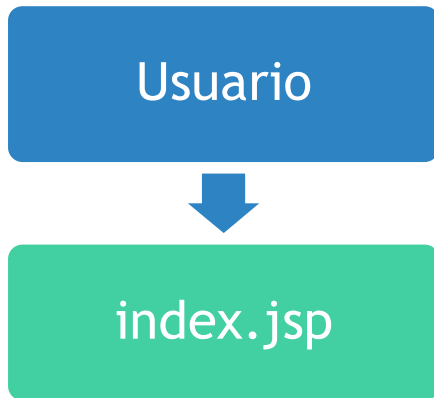
- ▶ HTML
- ▶ lógica Java
- ▶ procesamiento

Puede volver las aplicaciones difíciles de mantener.

Los servlets permiten separar el procesamiento de la presentación, dejando al JSP principalmente como componente visual evitando mezclar lógica de procesamiento con presentación.

¿Quién debería recibir primero la solicitud?

Caso 1 — Presentación simple



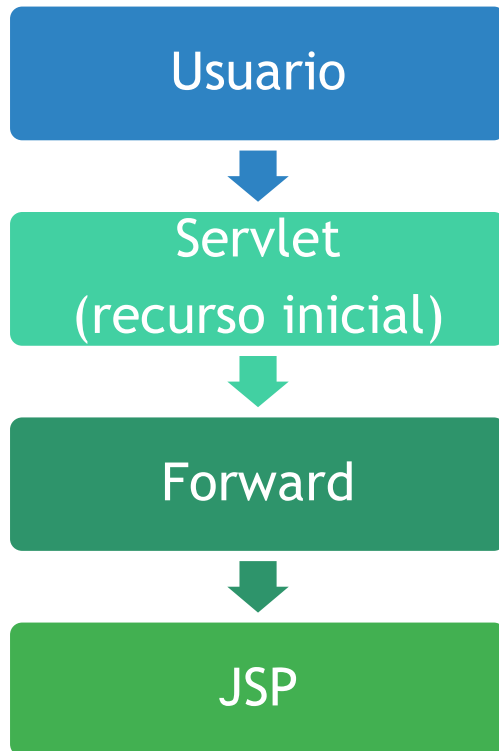
El JSP responde directamente.

Utilizado cuando:

- ▶ no existe procesamiento previo
- ▶ la información es estática o simple
- ▶ la vista no necesita preparar datos

¿Quién debería recibir primero la solicitud? (Cont.)

Caso 2 — Procesamiento previo



El servlet recibe la solicitud, procesa la información y luego delega la presentación al JSP.

Utilizado cuando:

- ▶ se requieren datos dinámicos
- ▶ validaciones
- ▶ procesamiento
- ▶ acceso a lógica de negocio

Toda aplicación Web necesita un punto inicial de acceso

Generalmente, cuando un usuario accede a una aplicación sin indicar un recurso específico: <https://www.ubp.edu.ar/>

El servidor busca automáticamente un archivo de bienvenida (*welcome file*), como, por ejemplo:

- ▶ index.jsp
- ▶ index.html

Pero... en aplicaciones más dinámicas, ese punto de entrada puede resolverse mediante un servlet.

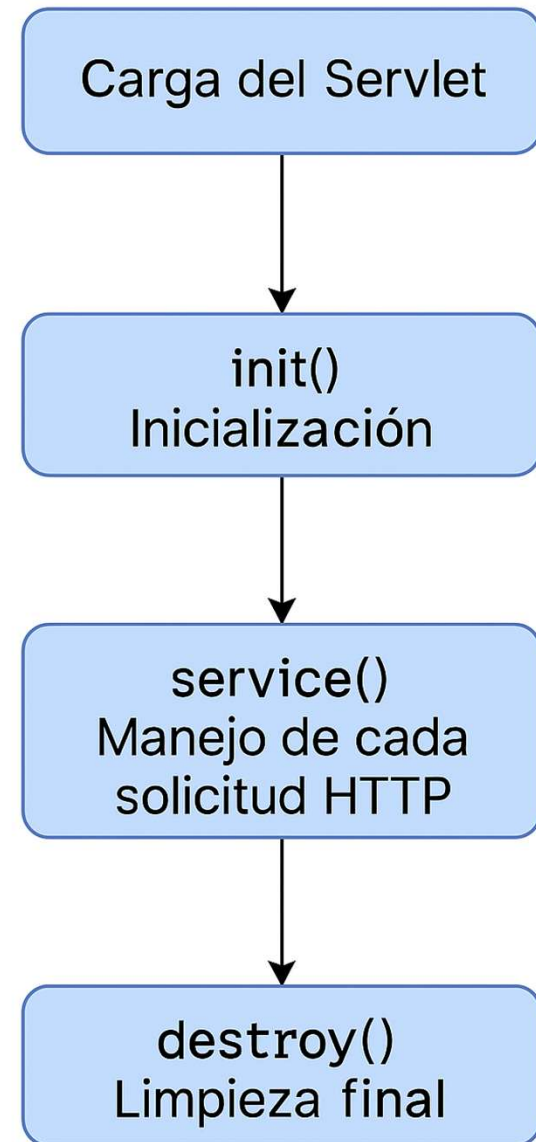
Es decir, el servlet pasa a responder como recurso inicial de la aplicación, tomando el rol que normalmente cumpliría un index.jsp.

Ciclo de vida de un Servlet

El ciclo de vida de un servlet es administrado por el contenedor Web (Tomcat, Jetty, GlassFish, etc.).

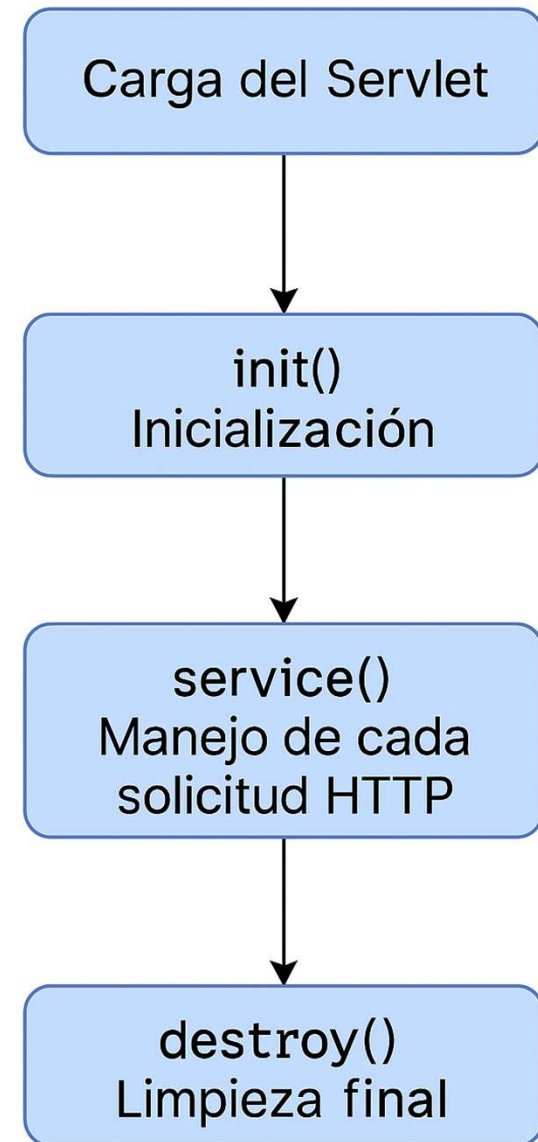
Un servlet:

- ▶ se carga una vez
- ▶ puede atender múltiples solicitudes
- ▶ permanece en memoria hasta que la aplicación finaliza o se reinicia



Etapas principales

- ▶ **init():** Se ejecuta una sola vez al inicializar el servlet. Se utiliza para preparar recursos necesarios.
- ▶ **service():** Se ejecuta por cada solicitud recibida. Determina el tipo de petición y delega en métodos como:
 - ▶ doGet()
 - ▶ doPost()
- ▶ **destroy():** Se ejecuta al finalizar el ciclo de vida del servlet. Permite liberar recursos utilizados.



Un mismo servlet puede atender múltiples usuarios

El contenedor reutiliza una misma instancia del servlet para procesar múltiples solicitudes simultáneamente.

Por este motivo:

- ▶ no se recomienda almacenar información específica del usuario en atributos de instancia
- ▶ cada solicitud debe manejar su propia información mediante request y response

HttpServletRequest

Representa la solicitud que el cliente (generalmente un navegador Web) hace a un servlet. Contiene toda la información relevante sobre la solicitud HTTP, incluyendo parámetros, encabezados, cookies y más. Algunos métodos importantes:

- ▶ **String getParameter(String name):** Devuelve el valor del parámetro de solicitud con el nombre especificado. Si el parámetro no existe, devuelve null.
- ▶ **String[] getParameterValues(String name):** Devuelve un array de String que contiene todos los valores del parámetro de solicitud con el nombre especificado. Si el parámetro no existe, devuelve null.

HttpServletRequest (Cont.)

Otros métodos importantes:

- ▶ **String getHeader(String name):** Devuelve el valor del encabezado de solicitud con el nombre especificado.
- ▶ **HttpSession getSession():** Devuelve la sesión actual asociada con esta solicitud, o crea una si no existe.

HttpServletResponse

Representa la respuesta que el servlet envía de vuelta al cliente. Contiene métodos para controlar y personalizar la respuesta HTTP, incluyendo el

establecimiento de códigos de estado, encabezados y contenido de la respuesta. Algunos métodos importantes:

- ▶ **void setContentType(String type):** Establece el tipo MIME de la respuesta.
- ▶ **PrintWriter getWriter():** Devuelve un `PrintWriter` que puede ser usado para enviar texto al cliente.
- ▶ **void setStatus(int sc):** Establece el código de estado de la respuesta.

HttpServletResponse (Cont.)

Otros métodos importantes:

- ▶ **void addHeader(String name, String value):** Añade un encabezado a la respuesta.
- ▶ **void addCookie(Cookie cookie):** Añade una cookie a la respuesta.

Métodos HTTP frecuentes

doGet()

- ▶ utilizado para solicitar información
- ▶ los parámetros viajan en la URL
- ▶ suele utilizarse para consultas o navegación

doPost()

- ▶ utilizado para enviar información
- ▶ los datos viajan en el cuerpo de la solicitud
- ▶ suele utilizarse en formularios



Configuración de Servlets

Los servlets pueden configurarse mediante:

- ▶ el archivo Web.xml
- ▶ las anotaciones Java



Configuración mediante archivo Web.xml

El archivo Web.xml es el descriptor de despliegue estándar para aplicaciones Web Java. Configurar un servlet en este archivo implica agregar entradas XML que describen el servlet y su mapeo de URL.

```
<servlet>
    <servlet-name>FileUploadServlet</servlet-name>
    <servlet-class>ar.edu.ubp.pdc.FileUploadServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>FileUploadServlet</servlet-name>
    <url-pattern>/upload</url-pattern>
</servlet-mapping>
```

Configuración mediante anotaciones

Las anotaciones simplifican la configuración y reducen la necesidad de archivos XML extensos.

```
@WebServlet("/upload")
@MultipartConfig
public class FileUploadServlet extends HttpServlet {
    //Lógica del servlet
}
```

Idea clave

Los servlets representan el componente encargado del procesamiento dentro de una aplicación Web Java.

Permiten:

- ▶ recibir solicitudes HTTP
- ▶ procesar información
- ▶ coordinar la comunicación entre cliente y servidor
- ▶ generar respuestas dinámicas

Los servlets permiten centralizar el procesamiento antes de generar la respuesta final al usuario.



